

AWS + Docker Project Roadmap

A 6-project progression designed to test your understanding of AWS, Dockerfiles, and Docker Compose by making every project use a meaningfully different containerization pattern.

Main rule for this roadmap

The Dockerfile and docker-compose.yml pattern must not repeat blindly from one project to the next. Every project below changes the build logic, service relationships, networking style, runtime strategy, or AWS deployment shape so that you are forced to think, not copy.

How these 6 projects are different

Project	Dockerfile style	Compose focus	AWS focus
1	Basic single-stage frontend + backend Dockerfiles	App, DB, admin UI, bridge network	EC2 deployment
2	Multi-stage build for frontend artifact generation	Build-only/local preview service	S3 + CloudFront
3	Nginx reverse proxy + backend image separation	Multiple app replicas, proxy routing, env separation	ALB + Auto Scaling on EC2
4	Health-check aware runtime and logging-focused image	Health checks, restart policy, metrics/log shipping sidecar	CloudWatch
5	Containerized scheduled worker / utility image	One-off task containers, env injection, volumes for reports	Lambda + EventBridge + cost cleanup ideas
6	Production-style image flow prepared for ECR	Split environments, override files, image tagging strategy	ECR + EC2 + full deployment flow

Project 1 - Dockerized Full-Stack App on EC2

Project goal	Build a frontend + backend + MongoDB + Mongo Express stack locally, then deploy it on EC2.
Dockerfile challenge	Write two different Dockerfiles: one for frontend and one for backend. Use a standard single-stage build first so you clearly understand WORKDIR, COPY, RUN, EXPOSE, CMD, and dependency installation.
Compose challenge	Use docker-compose.yml with four services: frontend, backend, mongo, and mongo-express. Practice ports, depends_on, service names, environment variables, named volumes, and internal container-to-container communication.
AWS integration	Launch an EC2 instance, install Docker and Compose, copy the project, and run the application on the server.
What this tests	This checks whether you understand the basic flow of Dockerfiles, container networking, volume usage, and EC2 deployment.
Expected deliverables	Frontend Dockerfile, backend Dockerfile, one compose file, running application on EC2, short notes on how requests flow from browser to backend to MongoDB.

Project 2 - Frontend Build Pipeline with S3 and CloudFront

Project goal	Create a frontend project that is built inside Docker and then deployed as static files to S3 and served through CloudFront.
Dockerfile challenge	Use a multi-stage Dockerfile. Stage 1 builds the frontend assets. Stage 2 optionally serves them locally with Nginx for testing. This is different from Project 1 because now Docker is used as a build pipeline, not just as an app runtime.
Compose challenge	Use Compose for local preview only. One service builds the app, and another optional Nginx service serves the generated build artifacts. This teaches artifact-based workflows rather than multi-container database applications.
AWS integration	Upload build files to S3 and distribute them using CloudFront.
What this tests	This checks whether you understand multi-stage builds, build-vs-runtime separation, static hosting, and CDN delivery.
Expected deliverables	Multi-stage Dockerfile, preview compose file, S3 deployment steps, CloudFront distribution setup, notes explaining why this Dockerfile is different from Project 1.

Project 3 - Reverse Proxy Architecture with Load Balancer Thinking

Project goal	Deploy a backend application behind Nginx and structure the project as if it were production-ready for multiple app instances.
Dockerfile challenge	Write one Dockerfile for the backend service and another for Nginx reverse proxy configuration. The backend image should be separate from the proxy image so you understand responsibility separation.
Compose challenge	Compose should include nginx, backend, and database or cache if needed. Add environment file usage, custom networks, and multiple backend replicas if you can simulate them locally. This is different because Compose is now doing traffic routing and proxy orchestration.

AWS integration	Map the design to EC2 instances behind an Application Load Balancer and think through how multiple instances would serve the same containerized app.
What this tests	This checks whether you understand reverse proxy flow, service discovery, port hiding, internal-only services, and scaling patterns.
Expected deliverables	Backend Dockerfile, Nginx Dockerfile or Nginx image config, compose file with proxy routing, architecture diagram notes, and ALB mapping explanation.

Project 4 - CloudWatch Monitoring Stack for a Containerized App

Project goal	Run an application that produces logs and metrics, then wire monitoring concepts into the container setup.
Dockerfile challenge	Modify the application Dockerfile so that it is monitoring-friendly. Add proper startup command handling, runtime environment variables, and application log output to stdout/stderr so containers can be monitored correctly.
Compose challenge	Add health checks, restart policies, and optionally a log/metrics sidecar pattern. This is different because Compose is no longer only about launching services; it is now about resilience and observability behavior.
AWS integration	Use CloudWatch to inspect logs, create alarms, and simulate CPU or traffic spikes.
What this tests	This checks whether you understand logging best practices in containers, health checks, restart behavior, and monitoring flow on AWS.
Expected deliverables	Updated Dockerfile, compose with healthcheck and restart policy, CloudWatch alarm notes, and a short explanation of logs vs metrics.

Project 5 - Scheduled Worker and AWS Automation Project

Project goal	Build a utility or worker container that performs background jobs such as cleanup, backup, report generation, or maintenance tasks.
Dockerfile challenge	Create a Dockerfile for a worker image, such as a Python or Node maintenance script container. This is different because it is not a web server image. It should run a task and exit, or run scheduled logic.
Compose challenge	Use Compose to pass credentials and environment variables, mount folders for generated reports, and run one-off services. You can also use profiles or a separate compose file for task execution. This teaches non-web Compose usage.
AWS integration	Connect the concept to Lambda and EventBridge by designing a cleanup workflow for unused snapshots, old logs, or stale resources.
What this tests	This checks whether you understand task containers, scheduled automation ideas, environment injection, and operational workflows in AWS.
Expected deliverables	Worker Dockerfile, task-focused compose file, sample automation script, Lambda/EventBridge mapping notes, and explanation of why this differs from Projects 1 to 4.

Project 6 - ECR Deployment and Production-Like Compose Strategy

Project goal	Package your final app properly, push images to ECR, and deploy them with a more production-like structure.
Dockerfile challenge	Prepare images that are clean, tagged properly, and ready for registry push. Improve Dockerfiles by reducing image size, using .dockerignore correctly, and separating build-time and run-time concerns.
Compose challenge	Use a base compose file and an override strategy or separate dev/prod compose files. This is different because now Compose is being used as an environment management tool, not only a local run command.
AWS integration	Push images to ECR, pull them on EC2, and run the deployment. Optionally connect the frontend to CloudFront and backend to EC2 or ALB.
What this tests	This checks whether you understand image registry flow, tagging, deployment discipline, dev-vs-prod configuration, and production thinking.
Expected deliverables	Final optimized Dockerfiles, base compose plus override or prod compose, ECR push/pull flow, deployment checklist, and final architecture notes.

Interview revision angle

- Why does Project 1 use standard Dockerfiles while Project 2 uses multi-stage builds?
- Why is Docker different when serving static files compared with running a backend API?
- What problem does Nginx solve in Project 3?
- Why are health checks and restart policies important in Project 4?
- Why is a worker container different from a web application container in Project 5?
- Why should Project 6 separate development and production Compose strategy?
- How do ECR, EC2, CloudFront, CloudWatch, Lambda, and EventBridge fit into the deployment lifecycle?

Recommended build order

Days	Focus
1-2	Project 1 - Understand baseline Dockerfiles and Compose service networking
3-4	Project 2 - Learn multi-stage builds and static deployment to S3 + CloudFront
5-6	Project 3 - Learn reverse proxy and scaling design
7-8	Project 4 - Add monitoring, resilience, and logs/metrics thinking
9-10	Project 5 - Build automation and scheduled task thinking
11-14	Project 6 - Final deployment, ECR, and production-style packaging